```
  it "can move up" do
    second.move_up
    expect(first.reload.project_order).to eq(2)
    expect(second.reload.project_order).to eq(1)
  end
  it "can move down" do
    second.move_down
    expect(third.reload.project_order).to eq(2)
    expect(second.reload.project_order).to eq(3)
  end

  it "handles edge case moves up" do
    first.move_up
    expect(first.reload.project_order).to eq(1)
  end

  it "handles edge case moves down" do
    third.move_down
    expect(third.reload.project_order).to eq(3)
  end
end
```

That's more assertions than I would normally place in a single test, but they are closely related, so I think it reads best as a single group.

Here's the next step:

integration/02/app/models/task.rb

```
def first_in_project?
  return false unless project
  project.tasks.first == self
end

def last_in_project?
  return false unless project
  project.tasks.last == self
end
```

Since the project has defined the task association to be in order, you can assume that the order returned by `project.tasks` is what you expect.

Now you need to place that logic in the view template, which you can do by adding some code inside the loop in `app/views/projects/show.html.erb`. There's a decision point here, which is whether to send that link to the regular update method for `TasksController`, which already exists and would need to be changed, or to create new `up` and `down` controller actions. It's not strictly RESTful, but this isn't strictly a RESTful action. It's a shortcut to a RESTful action, which I'm okay with here because it's constrained.